CSCE 411: Design and Analysis of Algorithms (Honors)

Week 11: Input Representation and Undecidability

Date: November 5, 2026 *Samson Zhou*

1 Prime Factorization and Primality Testing

A positive integer $N \geq 2$ is *prime* if its only positive divisors are _____
An integer $N \geq 2$ that is not prime is called *composite*.

Two different problems

- **Prime factorization:** Given $N \geq 2$, find the prime numbers whose product is N, including repeated factors.

- **Primality testing:** Given $N \geq 2$, determine whether N is prime.

Example What should each problem output on $N = 91$? What about $N = 97$?

Question 1. *Which of the two problems is a decision problem as stated?*

A *Prime factorization*

B *Primality testing*

C *Both*

D *Neither*

Observation. A factorization can tell us whether a number is prime. A YES or NO answer about primality does not, by itself, supply the factors of a composite number.

A simple algorithm

We could test whether any integer between 2 and $\lfloor\sqrt{N}\rfloor$ divides N.

```
TrialDivision(N)
  if N < 2 then
      return NO
  end if
  d ← 2
  while d^2 ≤ N do
      if d divides N then
          return NO
      end if
      d ← d + 1
  end while
  return YES
```

Why is it correct? Suppose $N = ab$ with $a, b > 1$. Why must at least one of a and b be at most $\sqrt{N}$?

Question 2. *The algorithm tries at most $\sqrt{N}$ possible divisors. Does this show that primality testing has a polynomial-time algorithm?*

A *Yes, because $\sqrt{N}$ is bounded by a polynomial in N*

B *No, because the algorithm is incorrect*

C *We must first determine the length of the input*

To be clear. Starting at 1 would not help: 1 divides every integer. We are looking for a nontrivial divisor.

2 How Long Is the Input?

The value of an integer and the length of its representation are different quantities. Let N be the integer and n the number of bits used to write it in binary, without leading zeros.

Warm-up Write 13 in binary. How many bits did you use?

An n-bit positive integer satisfies

$$2^{n-1} \leq N < 2^n.$$

Therefore,

$$n = _____.$$

Return to trial division On a prime input, the algorithm tests all candidates through $\lfloor \sqrt{N} \rfloor$. Express the number of candidates in terms of the input length n:

$$\sqrt{N} = _____.$$

Is this polynomial or exponential in n?

A numerical example For a prime with 100 binary digits, the number of candidates is on the scale of 2^{50}, which is about 10^{15}. The input itself has only _____

Observation. Polynomial time means polynomial in the _____, not necessarily in the numerical value represented by the input.

What if we change the representation?

In unary, the integer N is represented by N copies of the symbol 1.

Question 3. *If N is given in unary, is trial division polynomial-time in the input length?*

A *Yes*

B *No*

C *Only if N is prime*

Explain why the answer differs from the binary case.

Does a slow algorithm mean a hard problem?

Question 4. *Does the running time of trial division prove that binary primality testing is not in* P*?*

A *Yes*

B *No, because a different algorithm might be faster*

Theorem 2.1 (Agrawal–Kayal–Saxena)**.** *Primality testing is in* _____

We will use this theorem without proving the algorithm. It concerns testing whether a number is prime; it does not itself give an algorithm for finding all prime factors.

We have been asking whether a problem has an efficient algorithm. Next, we will ask a more basic question: does every decision problem have an algorithm at all?

3 Decision problems and undecidability

A decision problem Q is *undecidable* if there cannot exist an algorithm that always halts and provides the correct YES or NO answer.

Question 5. *Say that Q is an NP-Complete problem. Which of the following statements is true?*

A *Q is undecidable*

B *If $P \neq NP$, then Q is undecidable, but we just do not know for sure yet*

C *Neither of the above is true*

4 Halting and the Halting Problem

Just as an integer can be represented by a string, the code of a program can also be represented by a string.
Let P represent some notion of a computer program and I represent some type of input that you give P. You can think of both as strings.

We say that P *halts* on I if

Example Let $P = \text{OddHello}$ be represented by the following code snippet.

```
OddHello(I)
  if the input is a string with an even number of letter characters (a-z) then
      while true do
          continue
      end while
  else
      return "hello!"
  end if
```

Question 6. *Does $\langle P, I \rangle$ halt when $I =$ "hello"?*

A *Yes*

B *Nope*

C *It depends*

D *There is no way of knowing*

Question 7. *Does $\langle P, I \rangle$ halt when $I =$ "hi"?*

A *Yes*

B *Nope*

C *It depends*

D *There is no way of knowing*

For the next question, suppose our fixed text representation of P contains 101 lowercase letters.

Question 8. *Does $\langle P, I \rangle$ halt when $I = P$?*

A *Yes*

B *Nope*

C *This question cannot have an answer*

Definition: The Halting Problem. The *halting problem* is a decision problem that takes in inputs of the form $\langle P, I \rangle$ and outputs

- YES

- NO

To be clear

- An instance of the halting problem is given by a

- An algorithm A solves the halting problem if

Theorem 4.1. *The Halting Problem is* _____

Proof. Suppose there is an algorithm H that always halts and correctly solves the halting problem. Thus,

- $H(M, I)$ returns YES if _____
- $H(M, I)$ returns NO if _____

We can represent the code of a program M as a string $\langle M \rangle$. This string can also be used as the input to a program.

A new program Using H, construct the following program.

OPPOSITE(z)
  if z is not a valid program description **then**
    return
  end if
  if $H(z, z)$ returns YES **then**
    while true **do**
      continue
    end while
  else
    return
  end if

Observation. Here $H(z, z)$ asks whether the program described by z halts when its input is the string z.

Running a program on its own description

Let $b = \langle \textsc{Opposite} \rangle$ be the description of our new program. What happens when we run $\textsc{Opposite}(b)$?

- Suppose $H(b, b)$ returns YES. What does H predict, and what does Opposite actually do?

- Suppose $H(b, b)$ returns NO. What does H predict, and what does Opposite actually do?

In either case, _____ Therefore, no such algorithm H exists.

What if we impose a deadline?

Question 9. *Given a program M, an input I, and a positive integer T, can an algorithm always decide whether M halts on I within T steps?*

A *Yes*

B *No, because the halting problem is undecidable*

C *Only if M eventually halts on I*

Next time, we will study how much memory a computation needs. Later, we will return to this deadline question and use it to prove a lower bound on running time.